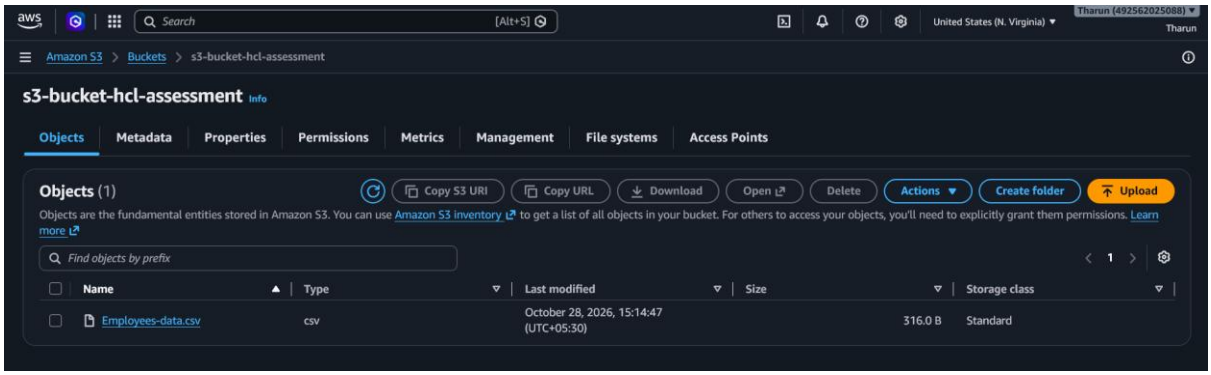
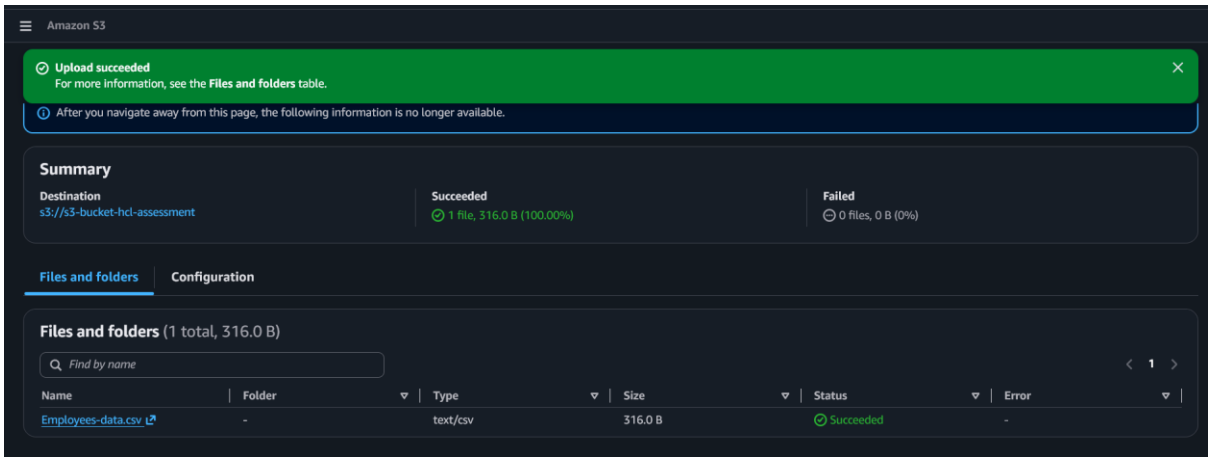
AWS CLOUD SERVICES

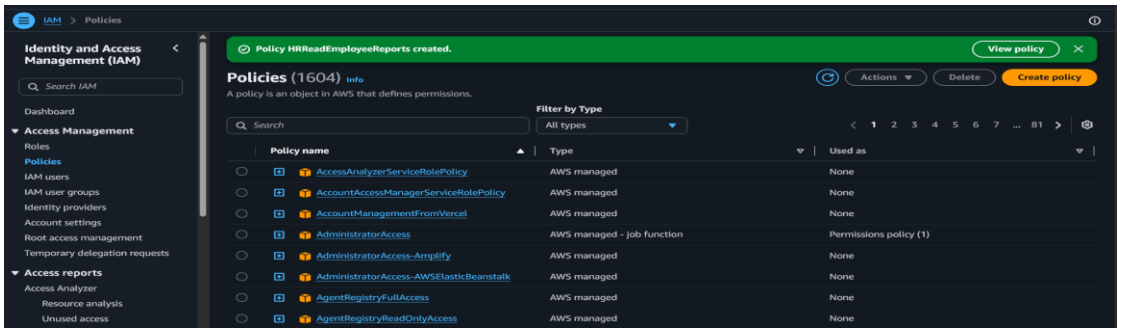
Question 10: S3 and IAM Security

a. Create an S3 bucket with Block Public Access enabled and upload a sample file.





b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



c. Explain how you would verify that an unauthorized user cannot read the file.

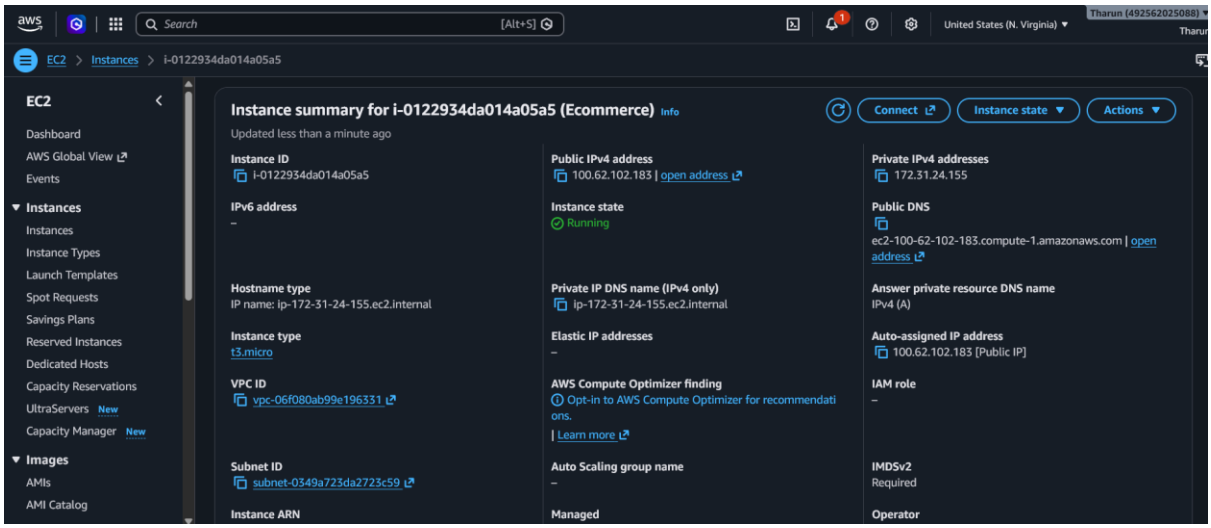
To verify that an unauthorized user cannot read the file, I would use an IAM user who does not have permission to access the S3 bucket. I would sign in as that user and attempt to open or download the uploaded file. AWS should display an “Access Denied” error, confirming that access is restricted to users with the required permissions.

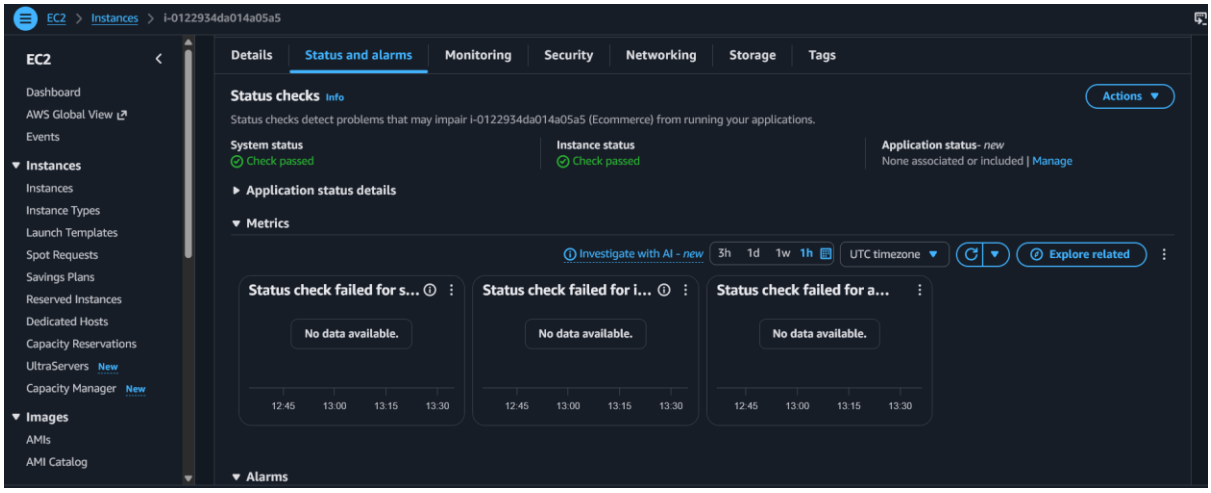
d. Explain how encryption at rest protects the stored data

I would enable default encryption on the S3 bucket to protect the stored employee reports. Amazon S3 uses server-side encryption to convert the stored data into an unreadable format, preventing unauthorized access to the underlying data. When an authorized user requests the file, Amazon S3 automatically decrypts it and provides access to the data.

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.





b. Explain how security groups and route tables can affect application connectivity.

Security Groups: I would check the inbound rules to ensure that HTTP (port 80) or HTTPS (port 443) traffic is allowed. Incorrect rules can prevent users from accessing the web application.

Route Tables: I would verify that the subnet has the correct routes. For a publicly accessible instance, the route table should have a route to an Internet Gateway, and the instance should have a public IP address.

Incorrect security group rules or route table configurations can cause connectivity problems.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

1. Open the Amazon CloudWatch Console.

2. Navigate to Alarms → All alarms → Create alarm.

3. Click Select metric → EC2 Metrics → Per-Instance Metrics.

4. Select the required EC2 instance and the CPUUtilization metric, then click Select metric.

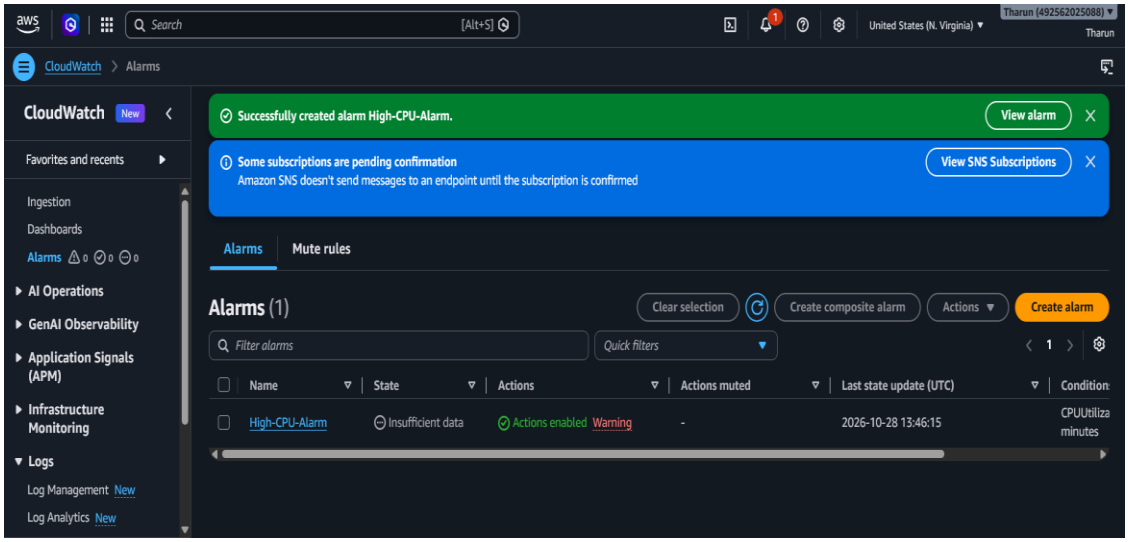
5. Set the statistic to Average and the period to 5 minutes.

6. Set the threshold condition to Greater than 85%.

7. Set the evaluation period to 2 consecutive periods so the alarm triggers when CPU utilization exceeds 85% for two consecutive five-minute periods.

8. Configure an optional Amazon SNS notification to receive an alert by email.

9. Name the alarm High-CPU-Alarm and click Create alarm.



d. Identify one method for investigating the root cause of high CPU utilization.

I would connect to the EC2 instance and use the Linux top command to monitor CPU usage and identify processes consuming excessive resources. I would then investigate the identified processes and check the application logs to determine the cause of high CPU utilization.

Question 12: Event-Driven Processing

a. Identify a suitable service for executing the processing code.

I would use AWS Lambda to execute the processing code. It can validate uploaded CSV files, process the data, and store the results in

a destination S3 bucket without requiring a server to be managed manually.

b. Explain how an S3 upload can trigger processing.

I would configure an S3 event notification to trigger the Lambda function whenever a new CSV file is uploaded. The Lambda function would then automatically read the uploaded file, validate its contents, process the data, and save the results to the destination location.

c. Explain the IAM permissions needed to read the input file and write the results.

I would create an IAM execution role for the Lambda function with the necessary permissions. The role would have s3:GetObject permission to read files from the input bucket and s3:PutObject permission to write processed results to the output bucket. These permissions would be restricted to the required resources to follow the principle of least privilege.

d. Explain how errors and processing logs can be captured.

I would use Amazon CloudWatch Logs to capture the Lambda function's execution logs, including processing details and error messages. If the function fails to process a CSV file, I could examine these logs to identify the cause and troubleshoot the issue.

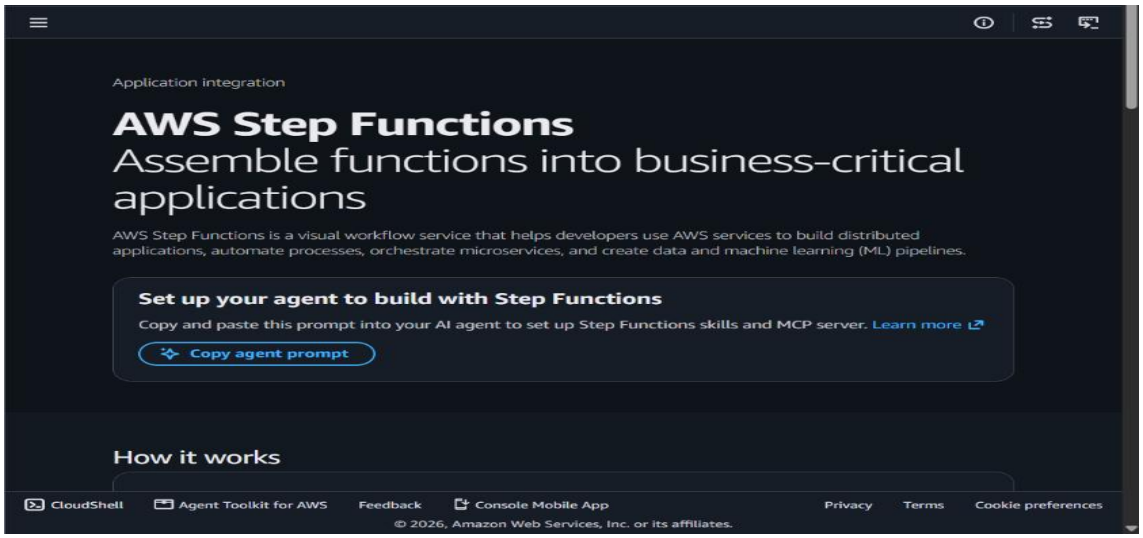
e. Explain how to prevent duplicate processing or recursive triggering.

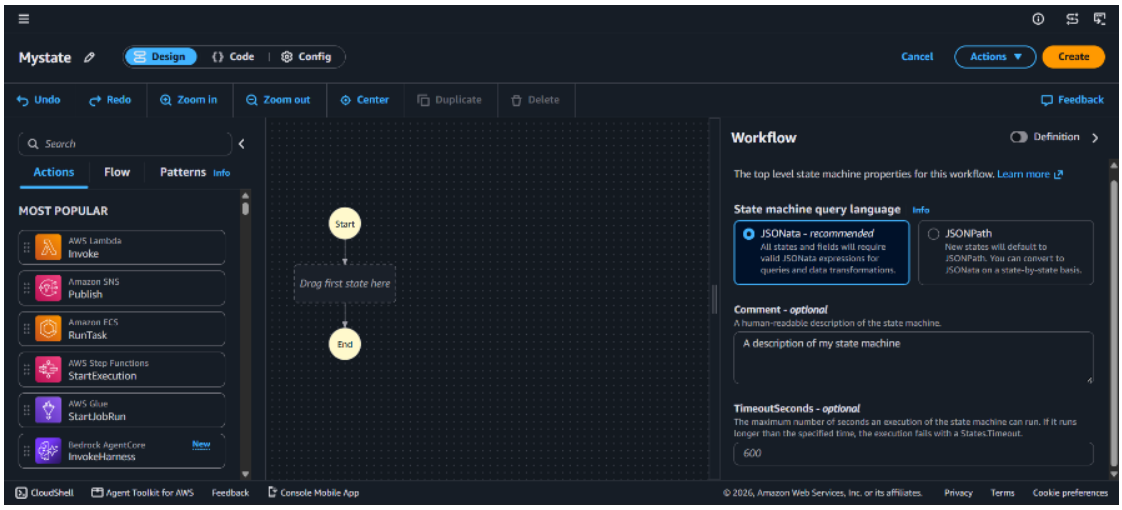
I would use separate S3 buckets for input files and processed results to prevent output uploads from triggering the same Lambda function again. To avoid duplicate processing, I would also track processed file names or version IDs using a service such as Amazon DynamoDB and skip files that have already been processed.

SECTION E

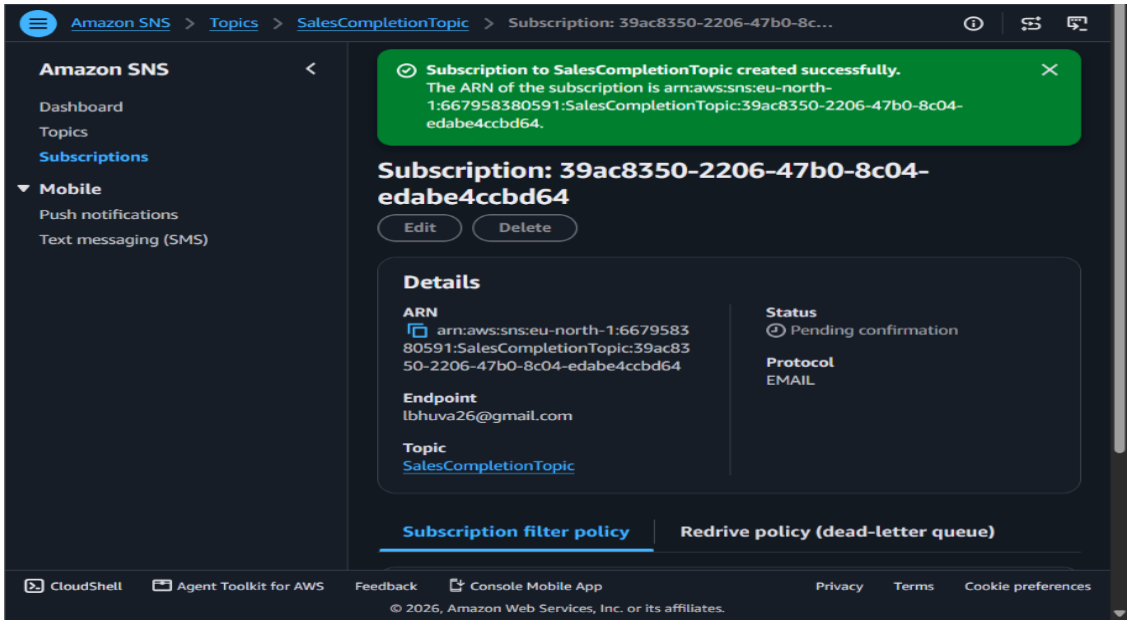
AWS STEP FUNCTIONS

Question 13: State Machine Design









a. Identify suitable states for validation, processing, storage, and notification.

I would use AWS Step Functions with the following states:

- ValidateInput: Validates the sales file using Lambda.
- CheckValidation: Routes valid files to processing and invalid files to failure handling.
- TransformData: Transforms the sales data.
- CalculateSalesByProduct: Calculates total sales for each product.
- StoreResults: Saves results to Amazon S3.
- SendCompletionNotification: Sends a completion notification.
- Success/Failure: Indicates whether the workflow completed or failed.

b. Explain how Retry and Catch can handle transient failures and permanent errors.

The Retry feature automatically retries a task when temporary errors occur, such as network issues or service failures. The Catch feature handles errors that continue after retries by redirecting the workflow to a failure-handling state, where the error can be logged and a notification can be sent.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

The Retry feature automatically retries tasks when temporary errors occur, such as service failures. The Catch feature handles errors that persist after retries by directing the workflow to a failure-handling state.

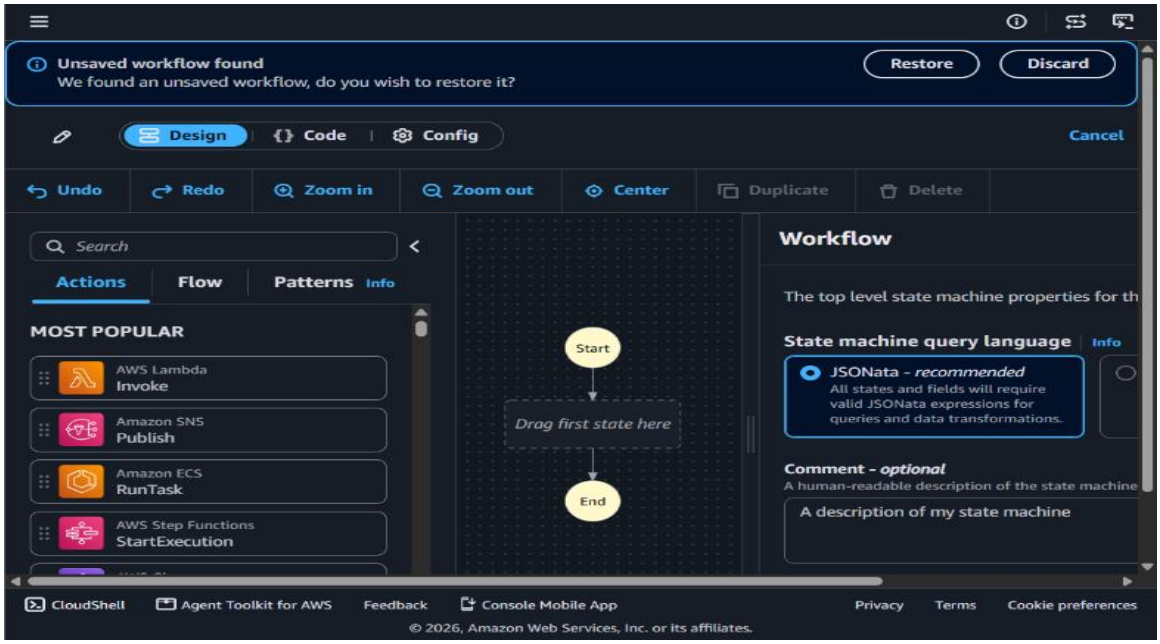
d. Explain how the state machine can invoke Lambda functions or other supported AWS services.

AWS Step Functions uses Task states to invoke Lambda functions or other supported AWS services. These tasks perform operations such as validating files, processing data, storing results in S3, and sending notifications.

e. Explain how execution history helps troubleshoot a failed workflow.

Execution history records the workflow's state transitions, inputs, outputs, and errors. It helps identify the step where the workflow failed and makes troubleshooting easier.

Question 14: Step Functions Implementation



(a) Three sequential tasks

The state machine uses three Task states: ValidateFile validates the input, TransformData transforms the data, and LoadResults loads the processed data into the destination. A Choice state checks the validation result and directs invalid files to a failure state.

(b) Retry policy

The TransformData task uses a Retry policy for temporary Lambda service errors and timeouts. It waits 2 seconds before the first retry, allows up to 3 retries, and doubles the waiting interval after each retry. If all retries fail, the workflow fails because no Catch handler is defined for this task.

AWS Step Functions

(c) Catch transition

The LoadResults task uses Catch with States.ALL to capture task errors and route them to RecordFailure. This state stores the error name and cause in the execution data before directing the workflow to WorkflowFailed. The recorded details are available in the Step Functions execution history.

(d) How data passes between tasks

AWS Step Functions passes the output of each state to the next state as input. The ResultPath field stores each task's result under a specified field, such as $.validation or $.transformation, while preserving the other input data. This allows subsequent tasks to access the earlier results.

(e) Testing successful and failed executions

I would test the state machine with a valid input file and verify that all three tasks execute successfully. I would then test an invalid file to confirm that the workflow stops, simulate a temporary transformation error to verify retries, and simulate a loading failure to verify that the Catch transition records the error. I would review the execution history to confirm the state transitions and outcomes.